Course Outline: AI Engineer Fundamentals

Title: AI Engineer Fundamentals: Your Introduction to AI-Assisted Development **Prerequisite:** None (Pre-work) **Target Audience:** Complete beginners starting their journey into AI-assisted software development **Total Lessons:** 17 **Estimated Total Length:** ~8,500 words **Format:** Conceptual with Guided Practice (35% hands-on)

Course Description

Welcome to the world of AI Engineering! This introductory course is designed for absolute beginners who want to understand how artificial intelligence can help them build software. You'll discover what coding agents are, how large language models work, and most importantly—how to work effectively alongside AI as your coding partner.

This is a gentle introduction to the tools and mindset you'll use throughout your bootcamp. You'll learn that AI doesn't replace you as a developer; instead, it amplifies your capabilities. By the end of this course, you'll understand how to leverage AI assistance while maintaining control over your code and development process.

This course connects directly to your upcoming bootcamp experience, where you'll use these AI tools daily to learn faster and build better projects.

Course Philosophy

This course emphasizes:

- **Human-in-the-loop approach:** You're always the decision-maker, AI is your assistant
- **Path of least resistance:** Start with simple, working solutions before optimizing
- **Vibecoding principles:** Iterate and improve rather than starting from scratch
- **Autonomous problem-solving:** Develop self-sufficiency while using AI tools
- **Attention to detail:** Cultivate careful reading and observation skills

---

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome to AI Engineering	1	~450
01 - How AI Helps You Code	3	~1,500
02 - Introduction to Coding Agents	3	~1,650
03 - Understanding LLMs Basics	3	~1,500
04 - AI Engineer Philosophy	3	~1,500
05 - Practice and Assessment	3	~1,950
06 - Your Journey Begins	1	~450
Total	**17**	**~8,500**

Section 00: Welcome to AI Engineering

00.0 Welcome to AI-Assisted Development 📖 (~450 words)

Learning Objectives:

- Understand what AI-assisted development means for beginners
- Recognize how AI tools will support your learning journey
- Set appropriate expectations for working with AI

Content Outline:

- What is AI-assisted development?
- Why AI tools matter for new developers
- Overview of your learning journey ahead
- How this course prepares you for the bootcamp

Transitions: This welcome lesson sets the stage for understanding AI as your coding partner, leading directly into how AI actually helps you write code.

Key Principles:

- AI augments human capabilities, doesn't replace them
- Learning to code with AI is learning to collaborate effectively
- You're entering development at an exciting time

Examples:

- Real-world scenario: A new developer using AI to understand error messages
- Comparison: Traditional coding vs. AI-assisted coding workflow

---

Section 01: How AI Helps You Code

01.0 AI as Your Coding Partner 📖 (~500 words)

Learning Objectives:

- Understand how AI functions as a development assistant
- Recognize different types of AI coding help
- Identify when to use AI assistance vs. independent problem-solving

Content Outline:

- How AI works in software development
- Types of help AI can provide (code generation, explanation, debugging)
- The collaborative relationship between developer and AI
- Setting boundaries: What AI can and cannot do

Transitions: Now that you understand the concept of AI as a partner, let's get hands-on and experience it firsthand.

Key Principles:

- AI provides suggestions, you make decisions
- Context is everything when working with AI
- Reading and understanding AI outputs is crucial

Examples:

- Example: Asking AI to explain a JavaScript function
- Example: Using AI to generate boilerplate code
- Example: Having AI help debug an error

01.1 Your First AI Coding Experience 💻 (~550 words)

Learning Objectives:

- Complete your first interaction with an AI coding assistant
- Practice writing clear requests to AI
- Learn to evaluate AI-generated code suggestions

Content Outline:

- Setting up access to a basic AI coding tool
- Writing your first prompt
- Understanding the response
- Making simple modifications

Transitions: Great! You've had your first AI coding experience. Now let's explore the landscape of available tools.

Key Principles:

- Clear communication leads to better AI responses
- Always read and understand what AI generates
- Small, specific requests work better than large, vague ones

Examples:

- Concrete example: Ask AI to create a simple "Hello World" program
- Example: Ask AI to explain what each line does
- Example: Modify the code with AI's help

Hands-On Exercise: Challenge: Your First AI Interaction

Task:

1. Access an AI coding assistant (provided by instructor)
2. Ask it: "Create a simple JavaScript program that prints 'Hello, AI Engineer!' to the console"
3. Read the generated code carefully
4. Ask the AI to explain what each line does
5. Modify the message to include your name

Success Criteria:

- Successfully generated code with AI assistance
- Can explain what the code does in your own words
- Made at least one modification to the code

01.2 Exploring Popular AI Tools 📖 (~450 words)

Learning Objectives:

- Identify major AI coding tools available today
- Understand differences between various AI assistants
- Learn which tools you'll use in the bootcamp

Content Outline:

- Overview of popular AI coding assistants (GitHub Copilot, ChatGPT, Claude, etc.)
- What makes each tool unique
- Introduction to 4Geeks' AI resources
- Choosing the right tool for different tasks

Transitions: You now know about various AI tools. Let's dive deeper into a specific category: Coding Agents.

Key Principles:

- Different AI tools have different strengths
- Some tools are specialized for coding, others are general-purpose
- You'll develop preferences as you gain experience

Examples:

- Comparison: GitHub Copilot (inline suggestions) vs. ChatGPT (conversational)
- Example: When to use a chat-based AI vs. an IDE-integrated tool
- Overview: Tools available at https://agents.4geeks.com/

Section 02: Introduction to Coding Agents

02.0 What Are Coding Agents? 📖 (~500 words)

Learning Objectives:

- Define what coding agents are and how they differ from basic AI assistants
- Understand the capabilities of modern coding agents
- Recognize when to use a coding agent vs. other AI tools

Content Outline:

- Definition: What makes something a "coding agent"
- How coding agents work in development environments
- Capabilities: What agents can do (file creation, editing, execution)
- Limitations: What agents can't do (decision-making, design choices)

Transitions: Now that you understand what coding agents are, let's set one up in your actual development environment.

Key Principles:

- Agents can perform actions, not just provide suggestions
- Agents work within your development environment
- You maintain oversight and control at all times

Examples:

- Example: Agent creating multiple files for a project structure
- Example: Agent running tests and interpreting results
- Contrast: Simple AI chat vs. coding agent capabilities

02.1 Setting Up a Coding Agent in GitHub Codespaces 💻 (~600 words)

Learning Objectives:

- Configure a basic coding agent in GitHub Codespaces
- Understand the components of agent setup
- Verify your agent is working correctly

Content Outline:

- What is GitHub Codespaces?
- Step-by-step agent installation
- Basic configuration settings

- Testing your agent setup
- Troubleshooting common issues

Transitions: Your agent is now configured! Let's explore the specific tools that 4Geeks provides.

Key Principles:

- Proper setup ensures smooth development experience
- Configuration can be adjusted as you learn
- Test your setup before starting major work

Examples:

- Walkthrough: Installing a coding agent extension
- Example: Configuring basic agent parameters
- Example: Running a test command to verify setup

Hands-On Exercise: Challenge: Configure Your First Coding Agent

Task:

1. Open GitHub Codespaces (instructor will provide access)
2. Follow the guided setup for a coding agent
3. Configure basic settings (model selection, response length)
4. Test the agent by asking it to create a simple file
5. Verify the agent can read and modify the file

Success Criteria:

- Agent successfully installed in Codespaces
- Created at least one file through agent commands
- Agent responds to basic coding requests

02.2 Tour of Agent Tools at 4Geeks 💻 (~550 words)

Learning Objectives:

- Navigate the 4Geeks agent tools platform
- Understand the different agents available to you
- Know how to access help and documentation

Content Outline:

- Introduction to https://agents.4geeks.com/
- Overview of available agents
- How to select the right agent for your task
- Accessing documentation and help resources

Transitions: You've now explored the agents available to you. To use them effectively, you need to understand how they actually work—which brings us to Large Language Models.

Key Principles:

- Different agents are optimized for different tasks
- Documentation is your friend when learning new tools
- The 4Geeks platform evolves with new agent capabilities

Examples:

- Demo: Navigating the agents.4geeks.com interface
- Example: Comparing two different agents for the same task

- Resource guide: Where to find help when stuck

Hands-On Exercise: Challenge: Explore Available Agents

Task:

1. Visit https://agents.4geeks.com/
2. Browse at least three different available agents
3. Read the description and capabilities of each
4. Identify which agent you'd use for: debugging, learning new concepts, and building a project
5. Bookmark the resources page

Success Criteria:

- Can navigate the agents platform independently
- Identified appropriate agents for different scenarios
- Knows where to find help documentation

Section 03: Understanding LLMs Basics

03.0 What Are Large Language Models? 📖 (~500 words)

Learning Objectives:

- Understand what Large Language Models (LLMs) are at a basic level
- Recognize how LLMs power the AI tools you're using
- Appreciate both capabilities and limitations of LLMs

Content Outline:

- Simple explanation: What is an LLM?
- How LLMs are trained (conceptual overview)
- Why LLMs can help with code
- What LLMs don't know (limitations and boundaries)

Transitions: Understanding what LLMs are helps you use them better. Now let's look at a practical aspect: how LLMs "measure" your requests and responses.

Key Principles:

- LLMs predict text based on patterns, they don't "understand" like humans
- Training data determines what LLMs know
- LLMs have knowledge cutoff dates
- Not all LLMs are the same

Examples:

- Analogy: LLMs are like extremely well-read assistants
- Example: How an LLM processes "Write a function to add two numbers"
- Example: What happens when you ask about very recent events

---

03.1 Tokens: How AI Counts Your Code 💻 (~550 words)

Learning Objectives:

- Understand what tokens are in LLM context
- Learn how token usage affects your AI interactions
- Develop awareness of efficient communication with AI

Content Outline:

- What is a token? (Simple definition)
- How text is broken into tokens
- Why tokens matter (cost and context limits)
- Practical tips for token-efficient prompts
- Viewing token counts in real tools

Transitions: Now that you understand tokens, let's explore other parameters that affect how LLMs respond to you.

Key Principles:

- Token efficiency = faster responses and lower costs
- More context isn't always better
- Being concise and clear saves tokens

Examples:

- Visual example: How "Hello, world!" breaks into tokens
- Comparison: Token-heavy vs. token-efficient prompts
- Calculator: Estimating tokens in your requests

Hands-On Exercise: Challenge: Token Awareness Exercise

Task:

1. Use a token counter tool (provided by instructor)
2. Type a simple coding request: "Create a function that adds two numbers"
3. Note the token count
4. Now type: "I need you to please create for me a JavaScript function that will take two numerical parameters and return their sum"
5. Compare token counts
6. Revise to be clear but concise

Success Criteria:

- Can estimate relative token usage
- Understands relationship between text length and token count
- Practiced writing concise, clear requests

03.2 Basic LLM Parameters 📖 (~450 words)

Learning Objectives:

- Understand key LLM parameters (temperature, max tokens)
- Recognize how parameters affect AI responses
- Know when to adjust parameters (basic awareness)

Content Outline:

- Overview of common LLM parameters
- Temperature: Creativity vs. consistency
- Max tokens: Response length limits
- Top-p and other parameters (brief mention)
- When to use default settings vs. custom

Transitions: You now understand the technical basics of how LLMs work. Let's shift to the philosophy and mindset of being an AI Engineer.

Key Principles:

- Default parameters work well for most cases
- Temperature affects response variety
- Understanding parameters helps you troubleshoot unexpected results

Examples:

- Example: Low temperature for code generation (consistent, predictable)
- Example: High temperature for brainstorming ideas (creative, varied)
- Comparison: Same prompt with different temperature settings

---

Section 04: AI Engineer Philosophy

04.0 Human-in-the-Loop: You're Still in Charge 📖 (~500 words)

Learning Objectives:

- Understand the human-in-the-loop development philosophy
- Recognize your role as the decision-maker
- Develop healthy skepticism about AI outputs

Content Outline:

- What does "human-in-the-loop" mean?
- Why AI needs human oversight
- Your responsibilities when using AI assistance
- Building good judgment about AI suggestions
- When to trust AI and when to question it

Transitions: Understanding your role as the human in the loop leads us to another key principle: choosing the simplest path that works.

Key Principles:

- AI is a tool, you're the craftsperson
- Every AI suggestion should be reviewed
- Building understanding is more important than generating code
- You're accountable for your code, not the AI

Examples:

- Scenario: AI suggests complex solution, you choose simpler approach
- Example: Reviewing AI-generated code before using it
- Case study: When blindly trusting AI leads to problems

04.1 The Path of Least Resistance 📖 (~500 words)

Learning Objectives:

- Understand the "path of least resistance" principle
- Learn to prioritize simple, working solutions
- Recognize when to add complexity vs. keep it simple

Content Outline:

- What is the path of least resistance?
- Why simple solutions are often best for beginners
- How to identify the simplest working approach
- When to add complexity (spoiler: later than you think)

- Applying this principle when working with AI

Transitions: Choosing simple solutions sets you up for the next principle: improving what works rather than starting over.

Key Principles:

- Working code beats perfect code
- Simple solutions are easier to understand and debug
- Complexity should be justified, not assumed
- AI can help you find simple solutions

Examples:

- Comparison: Simple for-loop vs. complex algorithm for small datasets
- Example: Using built-in functions before writing custom solutions
- Scenario: AI suggests advanced pattern, you request simpler alternative

---

04.2 Iterating vs. Starting Over (Vibecoding) 📖 (~500 words)

Learning Objectives:

- Understand the vibecoding philosophy
- Learn the value of iteration over perfection
- Develop comfort with incremental improvement

Content Outline:

- Introduction to vibecoding (vibemanifesto.org)
- Why iteration is powerful
- How to improve existing code vs. rewriting
- Recognizing when to iterate vs. when to start fresh
- Applying iteration mindset with AI tools

Transitions: These philosophical principles guide how you'll work throughout the bootcamp. Now let's put everything together in practice.

Key Principles:

- Iteration leads to better understanding than rewriting
- Every version teaches you something
- Perfect is the enemy of done
- AI makes iteration faster and easier

Examples:

- Example: Three iterations of a simple function, each better
- Contrast: Rewriting from scratch (slow) vs. iterating (fast)
- Real scenario: Improving AI-generated code through iterations
- Reference: Key concepts from vibemanifesto.org

Section 05: Practice and Assessment

05.0 Project: Excuse Generator with AI Assistance 💻 (~700 words)

Learning Objectives:

- Apply AI assistance to build a complete project
- Practice iterative development with AI

- Experience the full cycle from idea to working code

Content Outline:

- Project overview: Building a random excuse generator
- Breaking down the project into steps
- Working with AI agent step-by-step
- Testing and refining your code
- Celebrating your first AI-assisted project

Transitions: You've now built something real with AI assistance. Let's consolidate what you've learned about best practices.

Key Principles:

- Break projects into small, manageable steps
- Test each piece before moving forward
- Use AI for each step, but understand the result
- Working code is the first goal, improvement comes next

Examples:

- Step-by-step guide: Creating arrays with AI
- Example: Getting random elements from arrays
- Example: Combining elements into coherent output

Hands-On Exercise: Challenge: Build an Excuse Generator

Task:

1. Ask AI to create a JavaScript file
2. Request AI to create an array of "who" (e.g., "My dog", "My neighbor")
3. Request AI to create an array of "action" (e.g., "ate", "stole")
4. Request AI to create an array of "what" (e.g., "my homework", "my car keys")
5. Request AI to create an array of "when" (e.g., "yesterday", "this morning")
6. Ask AI to generate code that gets random values from each array
7. Request code to concatenate the values with spaces
8. Add a console.log to print the result

Code Example Structure:

```
 // Arrays created with AI assistance
const who = ["My dog", "My alarm clock", "A meteor"];
const action = ["ate", "destroyed", "stole"];
// ... etc.

// Random selection and concatenation
// AI helps generate this logic

// Output to console
console.log(excuse);
```

Success Criteria:

- Program generates random excuses
- Each excuse is grammatically coherent
- Code is well-commented (with AI's help)
- You understand what each line does
- Program runs without errors

05.1 Best Practices for AI-Assisted Development 📖 (~550 words)

Learning Objectives:

- Consolidate best practices learned throughout the course
- Recognize patterns for effective AI collaboration
- Prepare for ongoing AI-assisted learning

Content Outline:

- Key best practices review
- Common pitfalls and how to avoid them
- Building good habits from day one
- Continuous learning with AI tools
- Preparing for the bootcamp ahead

Transitions: You've learned the practices, built a project, and now it's time to assess your understanding.

Key Principles:

- Iterate, don't start from scratch (vibecoding)
- Keep humans in the loop at all times
- Choose simple solutions first
- Read and understand everything AI generates
- Ask questions and seek clarification

Examples:

- Best practice: Always review AI-generated code before running
- Best practice: Test in small increments
- Best practice: Save your prompts and successful approaches
- Anti-pattern: Copying AI code without understanding
- Anti-pattern: Asking AI to solve entire large problems at once

---

05.2 Knowledge Check: AI Engineering Basics 🧠 (~700 words)

Learning Objectives:

- Assess understanding of core AI engineering concepts
- Identify areas that need review
- Build confidence in foundational knowledge

Content Outline:

- Assessment overview and format
- Questions covering all course sections
- Scenario-based questions applying concepts
- Self-evaluation and reflection

Question Format: Scenario-Based Questions

Assessment Questions:

Question 1: Understanding AI Role You're working on a coding problem and the AI suggests a solution you don't fully understand. What should you do?

A) Copy the code and move on—AI knows best B) Ask the AI to explain the solution step-by-step, then decide if you'll use it C) Ignore the AI suggestion and struggle alone D) Ask a different AI and see if you get the same answer

Correct approach: B - Always seek understanding before using AI suggestions

Question 2: Token Efficiency You need to ask AI to create a simple login form. Which prompt is better?

A) "Please kindly create for me if you would be so kind a complete and comprehensive login form with absolutely all the features" B) "Create a login form with username, password, and submit button" C) "form" D) "I need a form but I'm not sure what kind maybe something for users to log in with their credentials"

Correct approach: B - Clear, concise, specific

Question 3: Path of Least Resistance You need to find the largest number in a small list of 5 numbers. AI suggests implementing a complex sorting algorithm. What should you do?

A) Accept the suggestion—more complex means better B) Ask for the simplest solution that works for this specific case C) Write no code—manually check the five numbers D) Insist on learning the complex algorithm first

Correct approach: B - Simple solution for simple problem

Question 4: Human-in-the-Loop The AI generates code that runs without errors, but you notice it doesn't handle negative numbers, which your project needs. What do you do?

A) Accept it—it runs without errors B) Identify the limitation and ask AI to modify the code to handle negative numbers C) Start completely over with a new approach D) Assume AI tested it thoroughly, so negative numbers must not matter

Correct approach: B - You're the decision-maker who understands requirements

Question 5: Iteration vs. Starting Over You have working code that's a bit messy. You want to improve it. What's the vibecoding approach?

A) Delete everything and start fresh for perfection B) Iterate on the working code, improving it step by step C) Leave it as-is—working is good enough forever D) Ask AI to rewrite the entire thing from scratch

Correct approach: B - Iterate and improve what works

Question 6: Coding Agent Capabilities What can a coding agent do that a simple AI chat assistant cannot?

A) Understand your questions better B) Create and modify files directly in your development environment C) Generate better code D) Work without internet connection

Correct approach: B - Agents can take actions in your environment

Question 7: Real-World Application You're building your first website and encounter an error message you don't understand. How should you use AI?

A) Copy the entire error into AI and ask it to fix everything B) Copy the error, ask AI to explain what it means, then decide how to fix it C) Avoid using AI—you should figure it out alone D) Restart your computer and hope the error goes away

Correct approach: B - Use AI to understand, then you take action

Evaluation Criteria:

- 6-7 correct: Excellent understanding of AI engineering principles
- 4-5 correct: Good grasp, review a few concepts
- 2-3 correct: Review course materials, especially human-in-the-loop and iteration
- 0-1 correct: Carefully review the entire course before proceeding

Score Interpretation: Understanding these principles will serve you throughout your bootcamp and career. If you scored lower than expected, revisit the specific sections related to questions you missed.

Section 06: Your Journey Begins

06.0 Ready for Your Bootcamp (~450 words)

Content Outline:

- Congratulations on completing AI Engineering Fundamentals
- Recap of key concepts: human-in-the-loop, path of least resistance, vibecoding
- How these principles apply throughout the bootcamp
- What to expect in Week 1 and beyond
- The tools and mindset you now have
- Encouragement: You're prepared to learn with AI assistance
- Resources for continued exploration
- Final thoughts: AI amplifies your capabilities as you learn to code

Note: This is a conclusion lesson only - no theory, exercises, or assessment.
